```
separate (IDL.SEM_PHASE)¶¶¬¬¬¬¬---------¶¬package body¬¬¬EXP_TYPE¶¬¬¬¬¬---------¶is¶¶  DEBUG¬¬: BOOLEAN¬¬:= FALSE;¶¶  use DEF_UTIL, VIS_UTIL;¶  use RED_
SUBP;¶  use REQ_UTIL;¶  use ATT_WALK;¶¶¬¬¬¬-----------------¶¬procedure¬¬¬REDUCE_EXP_TYPES¬¬¬(  DEFSET¬:DEFSET_TYPE;¶¬¬¬¬¬¬¬¬  TYPESET¬: out TYPESET_TY
PE)¶  is¶    TEMP_DEFSET¬: DEFSET_TYPE¬:= DEFSET;¶    DEFINTERP¬: DEFINTERP_TYPE;¶¶    NEW_TYPESET¬: TYPESET_TYPE¬:= EMPTY_TYPESET;¶    TYPE_SPEC¬: TR
EE;¶  begin¶    while¬not IS_EMPTY( TEMP_DEFSET ) loop¶      POP( TEMP_DEFSET, DEFINTERP );¶¶      TYPE_SPEC := GET_DEF_EXP_TYPE( GET_DEF( DEFINTERP )
 );¶¶      if TYPE_SPEC /= TREE_VOID then¶        ADD_TO_TYPESET( NEW_TYPESET, TYPE_SPEC, GET_EXTRAINFO( DEFINTERP ) );¶      end¬if;¶    end loop;¶
  TYPESET := NEW_TYPESET;¶¶  end¬REDUCE_EXP_TYPES;¶¬----------------¶¶¬¬¬¬-----------------------¶¬procedure¬¬¬REDUCE_DESIGNATED_TYPES¬¬( PREFIX_TYPES
ET :in out TYPESET_TYPE;¶¬¬¬¬¬¬¬¬  TYPESET¬        :out TYPESET_TYPE)¶  is¶    TEMP_PREFIXSET¬: TYPESET_TYPE¬:= PREFIX_TYPESET;¶    PREFIX_INTERP¬: TYP
EINTERP_TYPE;¶    PREFIX_TYPE¬: TREE;¶    PREFIX_STRUCT¬: TREE;¶    DESIG_TYPE¬: TREE;¶¶    NEW_PREFIXSET¬: TYPESET_TYPE¬:= EMPTY_TYPESET;¶    NEW_TYP
ESET¬: TYPESET_TYPE¬:= EMPTY_TYPESET;¶  begin¶    while¬not IS_EMPTY( TEMP_PREFIXSET ) loop¶      POP( TEMP_PREFIXSET, PREFIX_INTERP );¶      PREFIX_T
YPE¬:= GET_TYPE( PREFIX_INTERP );¶      PREFIX_STRUCT¬:= GET_BASE_STRUCT(¬PREFIX_TYPE );¶¶      if PREFIX_STRUCT.TY = DN_ACCESS then¶        DESIG_TYP
E := GET_BASE_TYPE( D(¬SM_DESIG_TYPE, PREFIX_STRUCT ) );¶        ADD_TO_TYPESET( NEW_PREFIXSET, PREFIX_INTERP);¶        ADD_TO_TYPESET( NEW_TYPESET, D
ESIG_TYPE, GET_EXTRAINFO( PREFIX_INTERP )¬);¶      end¬if;¶    end loop;¶¶    PREFIX_TYPESET := NEW_PREFIXSET;¶    TYPESET        := NEW_TYPESET;¶¶  e
nd¬REDUCE_DESIGNATED_TYPES;¶¬----------------------¶ ¶¶¶¬¬¬¬--==========--¶¬procedure¬¬¬EVAL_EXP_TYPES¬¬( EXP¬: TREE;¶¬¬¬¬¬¬¬  TYPESET¬:out TYPESET_T
YPE)¶  is¶    IS_SUBTYPE¬: BOOLEAN;¶  begin¶    EVAL_EXP_SUBTYPE_TYPES( EXP, TYPESET, IS_SUBTYPE );¶    if IS_SUBTYPE then¶      ERROR( D( LX_SRCPOS,
EXP ), "EXPRESSION (NOT SUBTYPE) REQUIRED"¬);¶      TYPESET := EMPTY_TYPESET;¶    end if;¶  end¬EVAL_EXP_TYPES;¶¬--===========--¶¶¬¬¬¬¬¬¬¬¬¶----------
--------------------------------------------------------------------------------------------------¶--¶--¶--¬¬¬procedure¬EVAL_EXP_SUBTYPE_TY
PES¬( EXP¬¬: TREE;¶--¬¬¬¬¬¬¬¬  TYPESET¬¬:out TYPESET_TYPE;¶--¬¬¬¬¬¬¬¬  IS_SUBTYPE_OUT¬:out BOOLEAN )¶--¶----------------------------------------------
--------------------------------------------------------------------------¶--¶--     Cette procedure verifie que EXP represente¬une expression, un sous-
type ou VOID.¶--     Au¬retour, TYPESET est¬l'ensemble des types de base possibles avec information implicite de conversion.¶--     IS_SUBTYPE_OUT est
 vrai si EXP reprsente un ou des sous-types¬; alors TYPESET fournit l'ensemble des¶--  types¬de base possible (il peut y en avoir plusieurs pour des f
ormes SIMPLE_EXPRESSION .. SIMPLE_EXPRESSION).¶--¶-----------------------------------------------------------------------------------------
--------------------¶¶¶¬¬¬¬--==================--¶¬procedure¬¬¬EVAL_EXP_SUBTYPE_TYPES¬( EXP¬¬: TREE;¶¬¬¬¬¬¬¬¬  TYPESET¬¬:out TYPESET_TYPE;¶¬¬¬¬¬¬¬¬  IS_
SUBTYPE_OUT¬:out BOOLEAN )¶  is¶¶    NEW_TYPESET¬: TYPESET_TYPE¬:= EMPTY_TYPESET;¶  begin¶    IS_SUBTYPE_OUT := FALSE;¶¶    if EXP = TREE_VOID then  T
YPESET :=¬EMPTY_TYPESET;  return;  end if;¶¶    if EXP.TY = DN_RANGE then¶      declare¶        EXP1¬: constant TREE¬:= D( AS_EXP1, EXP );¶        EXP
2¬: constant TREE¬:= D( AS_EXP2, EXP );¶        TYPESET_1¬: TYPESET_TYPE;¶        TYPESET_2¬: TYPESET_TYPE;¶      begin¶        EVAL_EXP_TYPES( EXP1,¬
TYPESET_1¬);¶        EVAL_EXP_TYPES( EXP2,¬TYPESET_2¬);¶        REQUIRE_SAME_TYPES( EXP1, TYPESET_1, EXP2, TYPESET_2, NEW_TYPESET );¶      end;¶¶
 IS_SUBTYPE_OUT := TRUE;¶      TYPESET¬ := NEW_TYPESET;¶      return;¶    end if;¶¶    if EXP.TY = DN_SUBTYPE_INDICATION then¶      declare¶        TY
PE_SPEC¬: TREE¬:= EVAL_SUBTYPE_INDICATION( EXP );¶      begin¶        if TYPE_SPEC /= TREE_VOID then¶¬ADD_TO_TYPESET( NEW_TYPESET, TYPE_SPEC );¶
  end if;¶        TYPESET := NEW_TYPESET;¶      end;¶¶      IS_SUBTYPE_OUT := TRUE;¶      return;¶    end if;¶¶    if EXP.TY not in CLASS_EXP then¶
   ERROR( D( LX_SRCPOS, EXP ), "EVAL_EXP_SUBTYPE_TYPES : EXPRESSION REQUIRED");¶      TYPESET := EMPTY_TYPESET;¶      return;¶    end if;¶¶    case CL
ASS_EXP'( EXP.TY )¬is¶¶      when DN_USED_OP | DN_USED_NAME_ID¬=>¶        PUT_LINE( "EVAL_EXP_SUBTYPE_TYPES : !! IMPOSSIBLE ARGUMENT FOR EVAL_EXP_TYPE
S" );¶        raise PROGRAM_ERROR;¶¶      when DN_ATTRIBUTE =>¶        EVAL_ATTRIBUTE( EXP, TYPESET, IS_SUBTYPE_OUT );¶        return;¶¶      when CLA
SS_USED_OBJECT | DN_SELECTED =>¶        declare¶¬DEFSET¬¬: DEFSET_TYPE;¶¬SOURCE_NAME¬: TREE;¶        begin¶¶if DEBUG then put_line( "EVAL_EXP_SUBTYPE_
TYPES CLASS_USED_OBJECT | DN_SELECTED" );¶  print_nod.print_node( EXP );¶end if;¶¶¬FIND_VISIBILITY( EXP, DEFSET );¶¬if not IS_EMPTY( DEFSET ) then¶¬
SOURCE_NAME := GET_THE_ID( DEFSET );¶¬  if SOURCE_NAME.TY¬in CLASS_TYPE_NAME¶¬¬and then not (¬GET_BASE_STRUCT( SOURCE_NAME ).TY = DN_TASK_SPEC¶¬¬¬¬and
 then DI( XD_LEX_LEVEL, GET_DEF_FOR_ID( D( XD_SOURCE_NAME, GET_BASE_STRUCT( SOURCE_NAME ) ) ) ) > 0¶¬¬¬    )¶¬  then¶¬    ADD_TO_TYPESET(¬NEW_TYPESET,
GET_BASE_TYPE( SOURCE_NAME¬) );¶¬    IS_SUBTYPE_OUT := TRUE;¶¬  else¶¬    REDUCE_EXP_TYPES( DEFSET,¬NEW_TYPESET );¶¬    if not IS_EMPTY¬( NEW_TYPESET
) then¶¬      if GET_THE_TYPE( NEW_TYPESET ).TY¬= DN_UNIVERSAL_INTEGER then¶¬        NEW_TYPESET¬:= EMPTY_TYPESET;¶¬        ADD_TO_TYPESET( NEW_TYPESE
T, MAKE( DN_ANY_INTEGER )¬);¶¬      elsif GET_THE_TYPE( NEW_TYPESET ).TY = DN_UNIVERSAL_REAL then¶¬        NEW_TYPESET¬:= EMPTY_TYPESET;¶¬        ADD_
TO_TYPESET( NEW_TYPESET, MAKE( DN_ANY_REAL ) );¶¬      end¬if;¶¬    end if;¶¬  end if;¶¬  STASH_DEFSET( EXP, DEFSET );¶¬end if;¶        end;¶¶      wh
en DN_FUNCTION_CALL =>¶        declare¶¬NAME¬¬: TREE¬:= D( AS_NAME, EXP );¶¬GENERAL_ASSOC_S¬: TREE¬:= D( AS_GENERAL_ASSOC_S, EXP¬);¶¶        begin¶¬ca
se CLASS_EXP'( NAME.TY ) is¶¬  when DN_ATTRIBUTE¬=>¶¬    EVAL_ATTRIBUTE(¬EXP, TYPESET, IS_SUBTYPE_OUT );¶¬    return;¶¶¬  when DN_USED_OBJECT_ID | DN_
SELECTED | DN_USED_OP =>¶¬    EVAL_SUBP_CALL(¬EXP, NEW_TYPESET );¶¶¬  when DN_STRING_LITERAL =>¶¬    NAME := MAKE_USED_OP_FROM_STRING (NAME);¶¬    D(
AS_NAME, EXP, NAME );¶¬    EVAL_SUBP_CALL(¬EXP, NEW_TYPESET );¶¶¬  when others =>¶¬    EVAL_SUBP_CALL(¬EXP, NEW_TYPESET );¶¬end case;¶        end;¶¶
    when DN_INDEXED | DN_SLICE =>¶        PUT_LINE( "EVAL_EXP_SUBTYPE_TYPES DN_INDEXED | DN_SLICE : IMPOSSIBLE ARGUMENT FOR EVAL_EXP_TYPES" );¶
 raise PROGRAM_ERROR;¶¶      when DN_ALL =>¶        declare¶¬NAME¬¬: constant TREE¬:= D( AS_NAME, EXP );¶¬PREFIX_TYPESET¬: TYPESET_TYPE;¶        begin
¶¬EVAL_EXP_TYPES( NAME, PREFIX_TYPESET );¶¶¬if not IS_EMPTY( PREFIX_TYPESET ) then¶¬  REDUCE_DESIGNATED_TYPES( PREFIX_TYPESET, NEW_TYPESET );¶¶¬  if I
S_EMPTY( NEW_TYPESET ) then¶¬    ERROR¬(D( LX_SRCPOS, EXP ), "EVAL_EXP_SUBTYPE_TYPES : PREFIX OF .ALL NOT ACCESS");¶¬  end if;¶¬end if;¶¶¬STASH_TYPESE
T( NAME, PREFIX_TYPESET );¶        end;¶¶      when DN_SHORT_CIRCUIT =>¶        declare¶¬EXP1¬: constant TREE¬:= D( AS_EXP1, EXP );¶¬EXP2¬: constant T
REE¬:= D( AS_EXP2, EXP );¶¬TYPESET_1¬: TYPESET_TYPE;¶¬TYPESET_2¬: TYPESET_TYPE;¶        begin¶¬EVAL_EXP_TYPES( EXP1, TYPESET_1 );¶¬EVAL_EXP_TYPES( EXP

2, TYPESET_2 );¶¶¬REQUIRE_BOOLEAN_TYPE( EXP1, TYPESET_1 );¶¬REQUIRE_BOOLEAN_TYPE( EXP2, TYPESET_2 );¶¬REQUIRE_SAME_TYPES¬( EXP1, TYPESET_1, EXP2, TYPE
SET_2, NEW_TYPESET );¶        end;¶¶      when DN_NUMERIC_LITERAL¬=>¶        declare¶¬VALUE¬: TREE¬:= UARITH.U_VALUE( PRINT_NAME( D( LX_NUMREP, EXP )
) );¶        begin¶¬D( SM_VALUE, EXP, VALUE );¶¶¬if VALUE.PT = HI or¬else VALUE.TY = DN_NUM_VAL then¶¬  ADD_TO_TYPESET( NEW_TYPESET, MAKE( DN_ANY_INTE
GER ) );¶¬elsif VALUE.TY = DN_REAL_VAL then¶¬  ADD_TO_TYPESET( NEW_TYPESET, MAKE( DN_ANY_REAL ) );¶¬else¶¬  PUT_LINE( "EVAL_EXP_SUBTYPE_TYPES : VALUE.
TY INCORRECT" );¶¬  raise PROGRAM_ERROR;¶¬end if;¶¶        end;¶¶      when DN_NULL_ACCESS =>¶        ADD_TO_TYPESET( NEW_TYPESET, MAKE( DN_ANY_ACCESS
 ) );¶¶      when CLASS_MEMBERSHIP =>¶        ADD_TO_TYPESET( NEW_TYPESET, PREDEFINED_BOOLEAN );¶        D( SM_EXP_TYPE, EXP, PREDEFINED_BOOLEAN );¶¶
     when DN_CONVERSION =>¶        PUT_LINE( "EVAL_EXP_SUBTYPE_TYPES : IMPOSSIBLE ARGUMENT FOR EVAL_EXP_TYPES");¶        raise PROGRAM_ERROR;¶¶      w
hen DN_QUALIFIED =>¶        declare¶¬NAME¬: constant TREE¬:= D( AS_NAME, EXP );¶¬TYPE_SPEC¬: TREE¬¬:= EVAL_TYPE_MARK( NAME );¶        begin¶¬if TYPE_S
PEC /= TREE_VOID then¶¬  ADD_TO_TYPESET( NEW_TYPESET, TYPE_SPEC );¶¬end if;¶        end;¶¶      when DN_PARENTHESIZED =>¶        declare¶¬SUBEXP¬: con
stant TREE¬:= D( AS_EXP, EXP );¶        begin¶¬EVAL_EXP_TYPES( SUBEXP, NEW_TYPESET );¶        end;¶¶      when DN_AGGREGATE =>¶        ADD_TO_TYPESET(
 NEW_TYPESET, MAKE( DN_ANY_COMPOSITE¬) );¶¶      when DN_STRING_LITERAL =>¶        ADD_TO_TYPESET( NEW_TYPESET, MAKE( DN_ANY_STRING ) );¶¶      when D
N_QUALIFIED_ALLOCATOR =>¶        declare¶¬QUALIFIED¬¬: constant TREE¬:= D( AS_QUALIFIED,¬EXP );¶¬TEMP_TYPESET¬: TYPESET_TYPE;¶¬ANY_ACCESS_OF¬: TREE¬¬:
= MAKE( DN_ANY_ACCESS_OF );¶        begin¶¬EVAL_EXP_TYPES( QUALIFIED, TEMP_TYPESET¬);¶¬if not IS_EMPTY( TEMP_TYPESET¬) then¶¬  D( XD_ITEM, ANY_ACCESS_
OF, GET_THE_TYPE( TEMP_TYPESET ) );¶¬  ADD_TO_TYPESET( NEW_TYPESET, ANY_ACCESS_OF );¶¬end if;¶        end;¶¶      when DN_SUBTYPE_ALLOCATOR =>¶
 declare¶¬SUBTYPE_INDICATION¬: constant TREE¬:= D( AS_SUBTYPE_INDICATION, EXP );¶¬TYPE_SPEC¬¬: TREE¬¬:= EVAL_SUBTYPE_INDICATION( SUBTYPE_INDICATION );
¶¬ANY_ACCESS_OF¬: TREE¬¬:= MAKE( DN_ANY_ACCESS_OF );¶        begin¶¬if TYPE_SPEC /= TREE_VOID then¶¬  D( XD_ITEM, ANY_ACCESS_OF, TYPE_SPEC );¶¬  ADD_T
O_TYPESET( NEW_TYPESET, ANY_ACCESS_OF );¶¬end if;¶        end;¶¶    end case;¶¶    TYPESET := NEW_TYPESET;¶¶  end¬EVAL_EXP_SUBTYPE_TYPES;¶¬--=========
=========--¶¶¶¶¬¬¬¬--===========--¶¬function¬¬¬EVAL_TYPE_MARK¬¬( EXP : TREE ) return TREE¶  is¶    DEFSET¬: DEFSET_TYPE¬:= EMPTY_DEFSET;¶    TYPE_ID¬:
TREE¬¬:= TREE_VOID;¶  begin¶    if EXP.TY = DN_SUBTYPE_INDICATION then¶      if D( AS_CONSTRAINT, EXP ) /= TREE_VOID then¶        ERROR( D( LX_SRCPOS,
 EXP ), "EVAL_TYPE_MARK : TYPE MARK REQUIRED" );¶      end¬if;¶      return EVAL_TYPE_MARK( D( AS_NAME, EXP ) );¶    end if;¶¶    if EXP.TY = DN_USED_
OBJECT_ID then¶      FIND_DIRECT_VISIBILITY(¬EXP, DEFSET );¶    elsif¬EXP.TY = DN_SELECTED and then¬D( AS_DESIGNATOR, EXP ).TY = DN_USED_OBJECT_ID the
n¶      FIND_SELECTED_VISIBILITY( EXP, DEFSET );¶    else¶      ERROR( D( LX_SRCPOS, EXP ), "EVAL_TYPE_MARK : TYPE MARK REQUIRED" );¶      return TREE
_VOID;¶    end if;¶¶    TYPE_ID := GET_THE_ID( DEFSET );¶    if TYPE_ID.TY in CLASS_TYPE_NAME then¶      null;¶    elsif¬TYPE_ID /= TREE_VOID then¶
   ERROR( D( LX_SRCPOS, EXP ), "EVAL_TYPE_MARK : NOT A TYPE NAME - " & PRINT_NAME( D( LX_SYMREP,¬TYPE_ID )¬) );¶      TYPE_ID := TREE_VOID;¶    end if
;¶¶    if EXP.TY = DN_USED_OBJECT_ID then¶      D( SM_DEFN, EXP, TYPE_ID );¶    else¶      D( SM_DEFN, D( AS_DESIGNATOR, EXP¬), TYPE_ID );¶    end if;
¶¶    return GET_BASE_TYPE( TYPE_ID );¶¶  end¬EVAL_TYPE_MARK;¶¬--==========--¶¶¶¶¬¬¬--====================--¶¬function¬¬EVAL_SUBTYPE_INDICATION¬¬( EXP
: TREE ) return TREE¶  is¶    BASE_TYPE : TREE;¶  begin¶    if EXP.TY = DN_SUBTYPE_INDICATION then¶      BASE_TYPE := EVAL_TYPE_MARK (D (AS_NAME, EXP)
);¶¬¬    -- NOTE CONSTRAINT EVALUATED IN RESOLVE PASS¶      return BASE_TYPE;¶¶    elsif¬EXP.TY = DN_USED_OBJECT_ID or¬else EXP.TY = DN_SELECTED then¶
      return EVAL_TYPE_MARK (EXP);¶¶    else¶      PUT_LINE ("!! $$$$ NODE SHOULD BE SUBTYPE INDICATION");¶      raise PROGRAM_ERROR;¶    end if;¶  en
d¬EVAL_SUBTYPE_INDICATION;¶¬--====================--¶¶¶¶¬¬¬¬--=======--¶¬procedure¬¬¬EVAL_RANGE¬¬( EXP :TREE; TYPESET :out TYPESET_TYPE )¶  is¶  begin¶
   if EXP.TY = DN_RANGE then¶      declare¶        EXP_1¬: constant TREE¬:= D( AS_EXP1, EXP );¶        EXP_2¬: constant TREE¬:= D( AS_EXP2, EXP );¶
     TYPESET_1¬: TYPESET_TYPE;¶        TYPESET_2¬: TYPESET_TYPE;¶      begin¶        EVAL_EXP_TYPES( EXP_1, TYPESET_1 );¶        EVAL_EXP_TYPES( EXP_2
, TYPESET_2 );¶        REQUIRE_SCALAR_TYPE( EXP_1, TYPESET_1 );¶        REQUIRE_SCALAR_TYPE( EXP_2, TYPESET_2 );¶        REQUIRE_SAME_TYPES ( EXP_1, T
YPESET_1, EXP_2, TYPESET_2, TYPESET );¶      end;¶¶    elsif¬EXP.TY = DN_ATTRIBUTE or else¬(EXP.TY =¬DN_FUNCTION_CALL and then D( AS_NAME, EXP ).TY =
DN_ATTRIBUTE ) then¶      declare¶        IS_SUBTYPE¬: BOOLEAN;¶      begin¶        EVAL_ATTRIBUTE( EXP, TYPESET, IS_SUBTYPE );¶        if not IS_SUBT
YPE then¶¬TYPESET := EMPTY_TYPESET;¶¬ERROR( D(¬LX_SRCPOS, EXP ), "RANGE ATTRIBUTE REQUIRED" );¶¶        end if;¶      end;¶    else¶      TYPESET := E
MPTY_TYPESET;¶      ERROR( D( LX_SRCPOS, EXP ), "RANGE REQUIRED" );¶    end if;¶  end¬EVAL_RANGE;¶¬--=======--¶¶¶¶¬¬¬¬--===============--¶¬procedure¬¬¬
EVAL_DISCRETE_RANGE¬¬( EXP :TREE; TYPESET :out TYPESET_TYPE)¶  is¶    NEW_TYPESET¬: TYPESET_TYPE;¶  begin¶    if EXP.TY = DN_RANGE or else EXP.TY¬= DN
_ATTRIBUTE or else EXP.TY¬= DN_FUNCTION_CALL¶    then¶      EVAL_RANGE( EXP, NEW_TYPESET );¶    else¶      declare¶        SUBTYPE_INDICATION¬: TREE;¶
        TYPE_SPEC¬¬: TREE;¶      begin¶        if EXP.TY =¬DN_DISCRETE_SUBTYPE¬then¶¬SUBTYPE_INDICATION := D( AS_SUBTYPE_INDICATION, EXP );¶        el
se¶¬SUBTYPE_INDICATION := EXP;¶        end if;¶¶        NEW_TYPESET¬:= EMPTY_TYPESET;¶        TYPE_SPEC¬:= EVAL_SUBTYPE_INDICATION( SUBTYPE_INDICATION
 );¶        if TYPE_SPEC /= TREE_VOID then¶¬ADD_TO_TYPESET( NEW_TYPESET, TYPE_SPEC );¶        end if;¶      end;¶    end if;¶¶    REQUIRE_DISCRETE_TYP
E( EXP, NEW_TYPESET );¶    TYPESET := NEW_TYPESET;¶¶  end¬EVAL_DISCRETE_RANGE;¶¬--================--¶¶¶¬¬¬¬--================================--¶¬procedure
¬¬¬EVAL_NON_UNIVERSAL_DISCRETE_RANGE¬( EXP :TREE;¶¬¬¬¬¬¬¬¬¬  TYPESET¬:out TYPESET_TYPE)¶  is¶¬      -- EVALUATE TYPES OF DISCRETE RANGE IN A CONTEXT WH
ERE¶¬      -- ... CONVERTIBLE UNIVERSAL_INTEGER IS TAKEN AS INTEGER¶¶    NEW_TYPESET : TYPESET_TYPE;¶    TYPE_NODE   : TREE;¶  begin¶    EVAL_DISCRETE
_RANGE( EXP,¬NEW_TYPESET );¶¶    if not IS_EMPTY( NEW_TYPESET )¶    then¶      TYPE_NODE := GET_THE_TYPE( NEW_TYPESET );¶      if TYPE_NODE.TY = DN_AN
Y_INTEGER and then EXP.TY = DN_RANGE and¬then D (AS_EXP1, EXP).TY /= DN_PARENTHESIZED and then D (AS_EXP2, EXP).TY /= DN_PARENTHESIZED then¶¶¬¬¬  -- R
EPLACE WITH PREDEFINED INTEGER¶        NEW_TYPESET¬:= EMPTY_TYPESET;¶        ADD_TO_TYPESET( NEW_TYPESET, PREDEFINED_INTEGER );¶      else¶        REQ
UIRE_NON_UNIVERSAL_TYPE( EXP, NEW_TYPESET );¶      end¬if;¶    end if;¶¶    TYPESET := NEW_TYPESET;¶¶  end¬EVAL_NON_UNIVERSAL_DISCRETE_RANGE;¶¬--=====
========================--¶¶¶¬---------¶end¬EXP_TYPE;¶¬---------¶
```